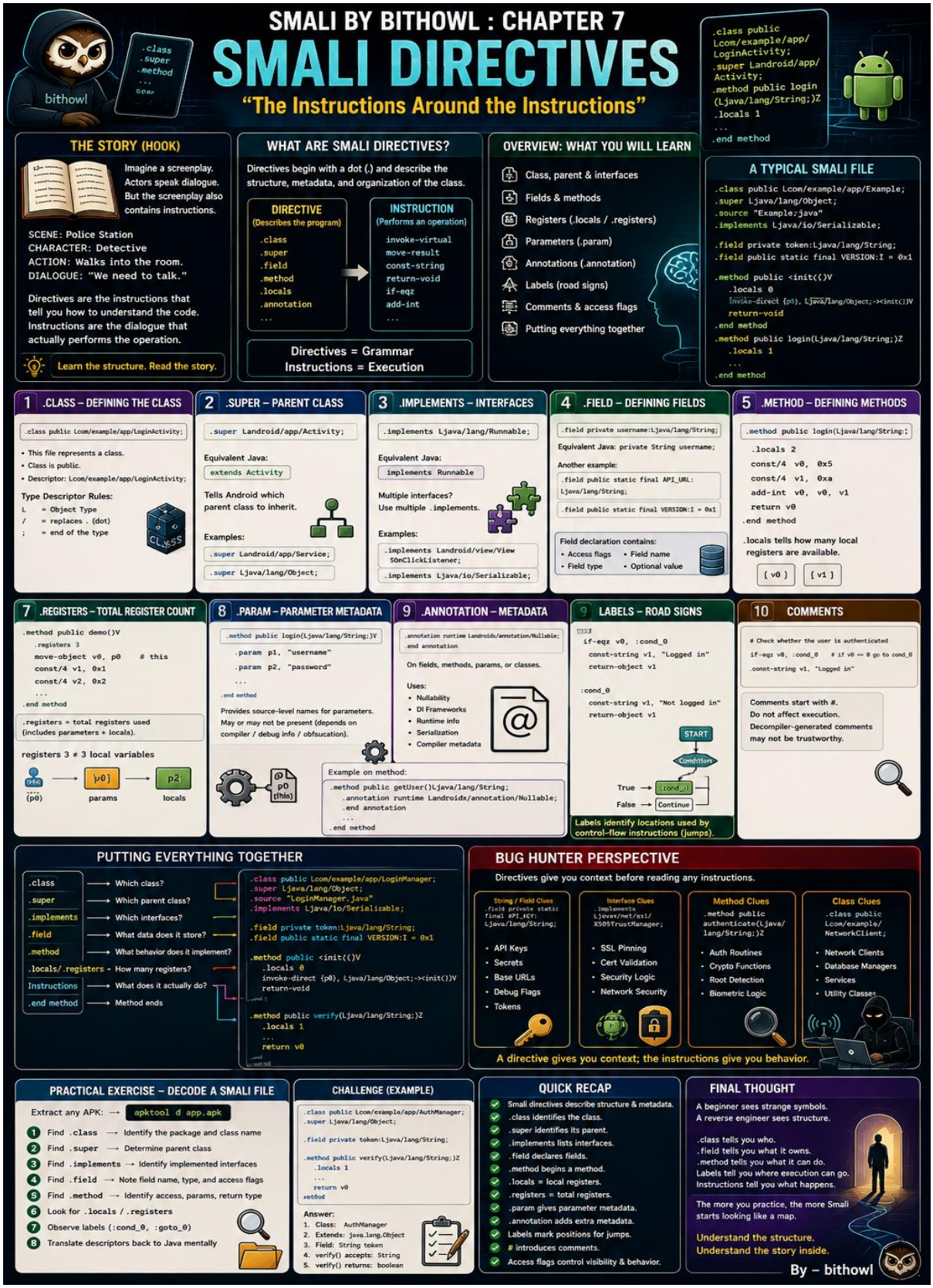
Smali By bithowl: Chapter 7 Smali Directives

("The Instructions Around the Instructions")



You've learned how to recognize a Smali class. You've learned how Java types transform into descriptors. Now it's time to understand the lines that tell Smali **what everything means**.

Consider this:

```
.class public Lcom/example/app/LoginActivity;

.super Landroid/app/Activity;

.source "LoginActivity.java"


.field private token:Ljava/lang/String;


.method public login(Ljava/lang/String;)Z
    .locals 1

    ...
.end method
```

If you've never seen Smali before, this looks like a wall of strange syntax. But look closer.

There is a pattern.

.class tells us what the class is.

.super tells us what it inherits.

.field describes data.

.method defines behavior.

.locals describes register allocation.

These are called **Smali directives**. They don't represent normal executable instructions. Instead, they describe the structure, metadata, and organization of the bytecode. Think of them as the **grammar surrounding the actual code**.

 # The Story (Hook)

Imagine you're reading a screenplay. The actors speak dialogue.

But the screenplay also contains instructions:

SCENE: Police Station

CHARACTER: Detective

ACTION: Walks into the room.

DIALOGUE: "We need to talk."

The instructions aren't the dialogue itself. They tell you **how to understand the dialogue**.Smali works similarly.

Instructions such as:

invoke-virtual

move-result

return

represent executable operations.

But directives such as:

.class

.method

.field

.locals

.annotation

describe the structure and metadata surrounding those operations.

Once you learn to distinguish the two, Smali becomes much easier to read.

 # What Are Smali Directives?

Smali directives begin with a dot:

.class

.super

.field

.method

.locals

They provide information to the Smali assembler and describe the structure of the class.

A useful distinction is:

Directive

↓

Describes the program

Instruction

↓

Performs an operation

For example: .method public getToken()Ljava/lang/String;

defines a method.

But: return-object v0

actually performs an operation inside that method.

1) .class — Defining the Class

The .class directive identifies the class represented by the Smali file.

Example: .class public Lcom/example/app/LoginActivity;

This tells us:

- The class is public.
- Its descriptor is Lcom/example/app/LoginActivity;.

Equivalent Java: public class LoginActivity

The .class directive is normally the first major declaration you'll encounter in a Smali file.

2) .super — Defining the Parent Class

The .super directive tells us which class this class inherits from.

Example: .super Landroid/app/Activity;

Equivalent Java: extends Activity

Another example: .super Ljava/lang/Object;

Many classes ultimately inherit from java.lang.Object.

When reverse engineering an unfamiliar application, .super is useful because it immediately gives you a clue about the class's role.

For example: .super Landroid/app/Service;

strongly suggests you're looking at a Service.

3) .implements — Interfaces

A class can implement one or more interfaces. Smali represents this with .implements.

Example: .implements Ljava/lang/Runnable;

Equivalent Java: implements Runnable

A class can have multiple .implements directives:

.implements Landroid/view/View$OnClickListener;

.implements Ljava/io/Serializable;

For security researchers, interfaces can provide useful clues.

For example, seeing: .implements Ljavax/net/ssl/X509TrustManager;

should immediately make you interested in the class's certificate-validation logic. The directive itself isn't a vulnerability.

It's a **navigation clue**.

4) .field — Defining Fields

Fields represent data belonging to a class.

Example: .field private username:Ljava/lang/String;

Equivalent Java: private String username;

Another: .field private static final API_URL:Ljava/lang/String;

Equivalent Java: private static final String API_URL;

A field declaration contains information such as:

- Access modifiers
- Field name
- Field type
- Optional initial value

For example: .field public static final VERSION:I = 0x1

This declares a public static final integer field initialized to 1.

5) .method — Defining Methods

The .method directive starts a method definition.

Example: .method public login(Ljava/lang/String;)Z

From Chapter 6, we can decode this as:

Method: login

Parameter: String

Return type: boolean

The method continues until: .end method

Example:

```
.method public login(Ljava/lang/String;)Z
    .locals 1

    ...

    return v0
```

.end method

Everything between .method and .end method belongs to that method.

6) .locals — Local Register Count

Now we reach one of the most important concepts in Smali: **Registers.**

Consider: .method public calculate()I

.locals 2

The .locals directive indicates how many registers are allocated for local variables within the method.

These registers are commonly referenced as:

v0

v1

v2

...

For example:

.locals 2

const/4 v0, 0x5

const/4 v1, 0xa

add-int v0, v0, v1

return v0

The exact register layout becomes especially important when analyzing or modifying Smali.

We'll dedicate later chapters to registers because they are one of the foundations of Smali reverse engineering.

7) .registers — Total Register Count

Another directive you will encounter is: .registers 3

This specifies the **total number of registers used by the method**.

This is different from .locals.

A useful mental model is:

.registers

|

└— Total registers

.locals

    |

    └── Local registers

For instance, a non-static instance method has an implicit this reference, which occupies a parameter register.

Therefore: .registers 3 does not necessarily mean three local variables. This distinction becomes very important when reading method parameters and register usage.

8) .param — Parameter Metadata

Smali can also contain metadata describing method parameters.

Example:

.method public login(Ljava/lang/String;)V

  .param p1, "username"

  ...

.end method

This tells us that the parameter represented by p1 was associated with the source-level name username.

Depending on compiler output and debug information, parameter metadata may or may not be present.

Obfuscation can also make the names less useful. So don't assume every method will contain .param.

9) .annotation — Metadata About Code

Annotations provide additional metadata.

Example:

.annotation runtime Landroidx/annotation/Nullable;

.end annotation

Annotations can describe things such as:

- Nullability
- Dependency injection
- Framework behavior
- Serialization
- Compiler metadata
- Runtime-visible information

You may also encounter annotations attached to methods, fields, parameters, or classes.

For example:

.method public getUser()Ljava/lang/String;

    .annotation runtime Landroidx/annotation/Nullable;

    .end annotation

    ...

.end method

The annotation doesn't necessarily change the method's core bytecode.

Instead, it provides additional information that tools, frameworks, or runtime mechanisms may use.

10) Labels — The Road Signs of Smali

Now we move from directives to something you'll see constantly inside methods: **Labels.**

Example:

:cond_0

or:

:goto_1

Labels identify locations within a method. They're especially important for control flow.

Consider:

if-eqz v0, :cond_0

const-string v1, "Logged in"

return-object v1

:cond_0

const-string v1, "Not logged in"

return-object v1

The instruction:

if-eqz v0, :cond_0

means that execution may jump to the location marked:

:cond_0

Think of labels as road signs.

START

Understanding labels is essential for understanding control flow.

11) Comments

Smali supports comments using: #

Example:

```
# Check whether the user is authenticated
if-eqz v0, :cond_0
```

Comments don't affect execution.

They're there for humans.

However, when you are reverse engineering, don't assume comments are trustworthy.

Decompiler-generated or manually added comments may describe what someone *thinks* the code does.

The instructions themselves are what matter.

12) Access Flags

You've already seen access modifiers such as: public, private, protected, static, final

These are called **access flags**.

They describe the properties of classes, methods, and fields.

Common flags include:

Flag	Meaning
public	Accessible from anywhere
private	Accessible only within the declaring class
protected	Accessible according to Java protected-access rules
static	Belongs to the class rather than an instance
final	Cannot normally be overridden or reassigned
abstract	Has no implementation at that level
native	Implemented outside the DEX bytecode
synchronized	Method synchronization behaviour
interface	Declares an interface
Enum	Declares an Enum type

For example: .method private static final verify(Ljava/lang/String;)Z

can be read as: private, static, final, method

Name: verify

Parameter: String

Return: boolean

You don't need to memorize everything immediately.

The goal is to learn how to **decode the line**.

 # Putting Everything Together

Let's look at a simplified Smali file:

.class public Lcom/example/app/LoginManager;

.super Ljava/lang/Object;

.source "LoginManager.java"

.field private token:Ljava/lang/String;

.method public constructor <init>()V

```
    .locals 0

    invoke-direct {p0}, Ljava/lang/Object;-><init>()V

    return-void
.end method

.method public verify(Ljava/lang/String;)Z
    .locals 1

    ...

    return v0
.end method
```

We can now read the structure:

```
.class
    ↓
```

Which class?

```
.super
    ↓
```

Which parent?

```
.source
    ↓
```

Original source metadata

```
.field
    ↓
```

What data does it store?

.method

↓

What behavior does it implement?

.locals

↓

How many local registers?

Instructions

↓

What does it actually do?

.end method

↓

Method ends

This is the point where Smali starts becoming readable.

 # Bug Hunter Perspective

A bug hunter doesn't necessarily open a Smali file and read every line. That would be painfully inefficient on a large application. Instead, directives help build a quick map of the class.

Imagine finding:

.class public Lcom/example/security/TrustManager;

.implements Ljavax/net/ssl/X509TrustManager;

That's immediately interesting.

Or:

.field private static final API_KEY:Ljava/lang/String;

That's worth investigating.

Or:

.method public verify(Ljava/lang/String;)Z

That could be authentication, validation, or something completely harmless.

The important lesson is:

A directive gives you context; the instructions give you behaviour.

Never report a vulnerability merely because you found a suspicious class, field, or method name. Use it as a starting point for deeper analysis.

 # Practical Exercise — Decode a Smali File

Extract a test APK: apktool d app.apk

Pick one .smali file.

Don't use JADX yet.

Find:

Step 1 — Class

.class

Identify the package and class name.

Step 2 — Parent

.super

Determine what the class extends.

Step 3 — Interfaces

Search for:

.implements

Step 4 — Fields

Search for:

.field

Identify:

- Field name
- Type
- Access flags

Step 5 — Methods

Search for:

.method

For each method, identify:

- Access flags
- Method name
- Parameters
- Return type

- .locals or .registers

Step 6 — Control Flow

Inside a method, find:

:cond_0

:goto_0

:cond_1

Then find instructions that jump to those labels. You're beginning to reconstruct the method's control flow manually. That's reverse engineering.

A Small Challenge

Try decoding this:

.class public Lcom/example/app/AuthManager;

.super Ljava/lang/Object;

.field private token:Ljava/lang/String;

.method public verify(Ljava/lang/String;)Z

.locals 1

...

return v0

.end method

Without opening JADX, answer:

1. What is the class?

AuthManager

2. What does it extend?

java.lang.Object

3. What field does it contain?

String token

4. What does verify() accept?

One String

5. What does verify() return?

boolean

You're no longer just looking at Smali. You're decoding it.

 # Quick Recap

- Smali directives describe the structure and metadata of a class.
- .class identifies the class.
- .super identifies its parent.
- .implements identifies interfaces.
- .field declares fields.
- .method begins a method.
- .locals specifies the number of local registers.
- .registers specifies the total register count.
- .param provides parameter metadata.
- .annotation provides additional metadata.
- Labels such as :cond_0 mark locations used by control-flow instructions.
- # introduces comments.
- Access flags describe properties such as public, private, static, and final.

 # Final Thought

At the beginning of this journey, Smali looked like a secret language. Then you learned its file structure. Then its descriptors. Now you're learning its grammar.

.class tells you **who**.

.field tells you **what it owns**.

.method tells you **what it can do**.

Labels tell you **where execution can go**.

And the instructions tell you **what actually happens**.

The more of these pieces you recognize, the less Smali looks like gibberish.

Eventually, you'll open a Smali file…

…and start reading it like a map.

By -bithowl